

[31] DuckDB-VSS: Vector similarity search extension for DuckDB 총정리

저자: DuckDB Team

형태: 오픈소스 벡터 검색 확장 프로젝트

저장소: GitHub

공개/업데이트: 2024

요약

DuckDB-VSS는 DuckDB 데이터베이스에 벡터 유사도 검색(Vector Similarity Search) 기능을 추가하는 오픈소스 확장이다. DuckDB는 인메모리 컬럼 기반 OLAP 데이터베이스로 최근 빠르게 주목받는 시스템인데, DuckDB-VSS는 이 강력한 분석 엔진에 벡터 검색 기능을 통합한다. 이 확장은 정형 데이터와 벡터 데이터를 동시에 처리해야 하는 분석 워크로드를 지원한다. DuckDB의 컬럼 저장소 특성과 벡터 데이터의 조화로운 통합을 통해, 뛰어난 성능과 사용의 단순성을 모두 달성한다. 특히 프로토타이핑, 데이터 분석, 소규모에서 중규모의 벡터 검색이 필요한 시나리오에 적합하다.

상세분석

31.1 주요 문제점과 설계 동기

기존 벡터 검색 솔루션의 한계:

- **분석과 벡터 검색의 분리**: 기존 벡터 DB는 순수 벡터 검색에만 최적화, 정형 데이터와의 복합 분석이 어려움
- **OLAP 환경의 필요성**: 분석가가 벡터와 정형 데이터를 함께 분석해야 하는 경우 증가
- **진입 장벽**: 별도의 벡터 DB 시스템 관리의 복잡성
- **비용**: 중규모 조직이 벡터 DB를 위해 추가 인프라 투자의 부담
- **컬럼 저장소의 미활용**: DuckDB의 효율적 컬럼 저장소를 벡터 데이터에도 활용하지 못함

31.2 DuckDB의 기초

특징

컬럼 기반 저장:

- 행(row) 기반이 아닌 열(column) 기반 저장
- OLAP 분석 쿼리에 최적화
- 압축을 통한 저장소 절감
- 캐시 효율성 증대

인메모리 OLAP:

- SQL 쿼리 엔진 내장
- 복잡한 분석 쿼리 지원
- 매우 빠른 처리 속도

확장성:

- 모듈식 아키텍처
- 커스텀 함수, 타입, 연산자 추가 용이

31.3 DuckDB-VSS의 핵심 기능

벡터 데이터 타입

```
-- 벡터 컬럼 정의
CREATE TABLE embeddings (
  id INTEGER PRIMARY KEY,
  text VARCHAR,
  vector DOUBLE[1536] -- 1536차원 벡터
);
```

지원 형식:

- DOUBLE[]: 부동소수점 벡터
- FLOAT[]: 단정밀도 벡터
- 고정 길이 배열로 정의

벡터 거리 연산자

SQL 함수로 제공:

```
-- L2 거리 (유클리드 거리)
SELECT id, text,
       array_distance(vector, [1.2, 3.4, ...]) AS distance
FROM embeddings
ORDER BY distance
LIMIT 10;

-- 코사인 유사도
SELECT id, text,
       array_cosine_similarity(vector, [1.2, 3.4, ...]) AS similarity
FROM embeddings
ORDER BY similarity DESC
LIMIT 10;

-- 내적
SELECT id, text,
       array_dot_product(vector, [1.2, 3.4, ...]) AS dot
FROM embeddings
ORDER BY dot DESC
LIMIT 10;
```

지원하는 거리 메트릭:

- L2 (Euclidean)
- 코사인 (Cosine)
- 내적 (Dot Product)
- 맨해튼 (Manhattan)

벡터 인덱싱

HNSW (Hierarchical Navigable Small World) 인덱스:

```
-- 벡터 인덱스 생성
CREATE INDEX vec_idx ON embeddings
USING hnsw (vector)
WITH (distance = 'l2', m = 16, ef = 128);
```

파라미터:

- `distance`: 사용할 거리 메트릭
- `m`: HNSW 그래프의 최대 연결 수
- `ef`: 검색 시 탐색 효율성 파라미터 (클수록 정확하나 느림)

31.4 DuckDB-VSS의 아키텍처

저장소 계층

```

SQL 쿼리
  ↓
벡터 연산 플래너
  ↓
HNSW 인덱스 (선택적)
  ↓
컬럼 저장소 (벡터 데이터)
  ↓
메모리/디스크
  
```

컬럼 저장소의 장점:

- 벡터 차원 단위의 저장으로 캐시 효율성
- 배치 처리에 최적화
- SIMD 명령어 활용 용이

쿼리 처리

선택형 HNSW 가속:

- 인덱스가 있으면 HNSW로 후보 선택
- 인덱스가 없으면 전체 스캔
- 쿼리 플래너가 인덱스 사용 여부 결정

벡터와 스칼라 조건의 결합:

```

SELECT id, text, category,
       array_distance(vector, query_vec) AS dist
FROM embeddings
WHERE category = 'news'           -- 스칼라 필터
      AND date >= '2024-01-01'   -- 추가 필터
ORDER BY dist
LIMIT 10;
  
```

처리 최적화:

1. 스칼라 필터 먼저 처리 (선택도 낮춤)
2. 결과에 대해 벡터 거리 계산
3. 거리 기반 정렬 및 제한

31.5 사용 사례

데이터 분석 (OLAP)

```
-- 임베딩된 뉴스 기사에서 특정 주제와 유사하면서
-- 특정 기간의 기사 분석
SELECT category, COUNT(*) as count,
       AVG(array_distance(embedding, topic_vec)) as avg_similarity
FROM articles
WHERE published_date >= '2024-01-01'
GROUP BY category
HAVING AVG(array_distance(embedding, topic_vec)) < 0.5
ORDER BY avg_similarity;
```

AI 애플리케이션 통합

```
import duckdb

conn = duckdb.connect()

# 벡터 데이터 로드
conn.execute("""
    CREATE TABLE documents AS
    SELECT doc_id, content, embedding
    FROM read_csv('docs.csv')
""")

# 검색 쿼리
query_vec = [0.1, 0.2, 0.3, ...]
results = conn.execute(f"""
    SELECT doc_id, content,
           array_distance(embedding, {query_vec}) as distance
    FROM documents
    ORDER BY distance
    LIMIT 10
""").fetchall()
```

ETL 파이프라인

```
-- 벡터 생성 및 저장
CREATE TABLE embeddings AS
SELECT id,
       text,
       ai_embed(text) as vector -- 커스텀 임베딩 함수
FROM raw_documents
WHERE valid = true;

-- 벡터 인덱싱
CREATE INDEX ON embeddings USING hnsw (vector);

-- 배치 검색
SELECT DISTINCT doc_a, doc_b
FROM (
    SELECT a.id as doc_a, b.id as doc_b,
           array_distance(a.vector, b.vector) as sim
    FROM embeddings a
    JOIN embeddings b ON a.category = b.category
) t
WHERE sim < 0.1
LIMIT 1000;
```

31.6 성능 특성

장점

빠른 벡터 검색:

- HNSW 인덱스로 대규모 데이터셋에서도 밀리초 단위 검색
- 컬럼 저장소로 배치 처리 최적화

메모리 효율성:

- 컬럼 압축으로 저장소 절감
- 불필요한 컬럼 제외로 메모리 사용 최소화

분석 능력:

- SQL의 모든 분석 기능 활용 가능
- 벡터 검색 결과의 추가 집계/분석 용이

개발 편의성:

- DuckDB 설치 후 확장만 추가
- SQL 기반 인터페이스로 학습곡선 낮음

제약

확장성 한계:

- 인메모리 시스템이므로 메모리 크기 제한
- 매우 대규모 벡터 데이터셋에는 부적합

분산 처리 미지원:

- 단일 노드 시스템
- 다중 노드 클러스터링 불가

31.7 본 논문과의 관계

Exqutor은 텍스트 쿼리를 벡터 기반 검색으로 변환하는 하이브리드 시스템이다. DuckDB-VSS는 다음의 측면에서 Exqutor과 관련:

1. **분석 기반 검색**: Exqutor이 벡터 검색을 OLAP 환경에서 수행해야 한다면, DuckDB-VSS의 컬럼 저장소와 분석 기능이 적합
2. **스칼라-벡터 결합**: Exqutor의 텍스트 필터와 벡터 검색의 조합은 DuckDB-VSS의 복합 쿼리 처리와 유사
3. **빠른 프로토타이핑**: Exqutor의 초기 구현과 성능 평가에 DuckDB-VSS 사용 가능

추가 제기 문제

1. **메모리 제약 극복**: 인메모리 시스템의 근본적 한계를 어떻게 극복할 것인가? 디스크 기반 벡터 저장에 성능에 미치는 영향은?
2. **HNSW 파라미터 자동화**: `m` 과 `ef` 같은 파라미터를 사용자가 수동으로 튜닝해야 하는데, 자동 최적화 방법은?
3. **분산 처리**: DuckDB의 장점을 유지하면서 분산 벡터 검색을 지원할 수 있는가? 단일 노드 제약을 어떻게 해결할 것인가?
4. **인덱스 유지보수**: 높은 빈도의 삽입 작업 중 HNSW 인덱스의 지속적 갱신이 성능에 미치는 영향은?
5. **다양한 벡터 차원 지원**: 모든 차원의 벡터에 대해 일정한 성능을 유지할 수 있는가? 극단적으로 높은 차원(100,000+)의 벡터는 지원 가능한가?
6. **비교 분석**: DuckDB-VSS와 Pinecone, Milvus 같은 전문 벡터 DB의 성능-비용 트레이드오프는?